CHAPTER

21 Design Problems

We have described a simple but very powerful and flexible protocol which provides for variation in individual network packet sizes, transmission failures, sequencing, flow control, and the creation and destruction of process-to-process associations.

— *"A Protocol for Packet Network Intercommunication,"* V. G. CERF AND R. E. KAHN, 1974

You may be asked in an interview how to go about creating a set of services or a larger system, possibly on top of an algorithm that you have designed. These problems are fairly open-ended, and many can be the starting point for a large software project.

In an interview setting when someone asks such a question, you should have a conversation in which you demonstrate an ability to think creatively, understand design trade-offs, and attack unfamiliar problems. You should sketch key data structures and algorithms, as well as the technology stack (programming language, libraries, OS, hardware, and services) that you would use to solve the problem.

The answers in this chapter are presented in this context—they are meant to be examples of good responses in an interview and are not comprehensive state-of-the-art solutions.

We review patterns that are useful for designing systems in Table 21.1. Some other things to keep in mind when designing a system are implementation time, extensibility, scalability, testability, security, internationalization, and IP issues.

Table 21.1: System design patterns.

Design principle	Key points
Decomposition	Split the functionality, architecture, and code into manageable, reusable components.
Parallelism	Decompose the problem into subproblems that can be solved independently on different machines.
Caching	Store computation and later look it up to save work.

DECOMPOSITION

Good decompositions are critical to successfully solving system-level design problems. Functionality, architecture, and code all benefit from decomposition.

For example, in our solution to designing a system for online advertising (Problem 21.13 on Page 388), we decompose the goals into categories based on the stake

Scanned by CamScanner

holders. We decompose the architecture itself into a front-end and a back-end. The front-end is divided into user management, web page design, reporting functionality, etc. The back-end is made up of middleware, storage, database, cron services, and algorithms for ranking ads. The design of TEX (Problem 21.6 on Page 382) and Connexus (Problem 21.12 on Page 388) also illustrate such decompositions.

Decomposing code is a hallmark of object-oriented programming. The subject of design patterns is concerned with finding good ways to achieve code-reuse. Broadly speaking, design patterns are grouped into creational, structural, and behavioral patterns. Many specific patterns are very natural—strategy objects, adapters, builders, etc., appear in a number of places in our codebase. Freeman *et al.*'s *"Head First Design Patterns"* is, in our opinion, the right place to study design patterns.

Parallelism

In the context of interview questions parallelism is useful when dealing with scale, i.e., when the problem is too large to fit on a single machine or would take an unacceptably long time on a single machine. The key insight you need to display is that you know how to decompose the problem so that

- each subproblem can be solved relatively independently, and
- the solution to the original problem can be efficiently constructed from solutions to the subproblems.

Efficiency is typically measured in terms of central processing unit (CPU) time, random access memory (RAM), network bandwidth, number of memory and database accesses, etc.

Consider the problem of sorting a petascale integer array. If we know the distribution of the numbers, the best approach would be to define equal-sized ranges of integers and send one range to one machine for sorting. The sorted numbers would just need to be concatenated in the correct order. If the distribution is not known then we can send equal-sized arbitrary subsets to each machine and then merge the sorted results, e.g., using a min-heap. Details are given in Solution 20.10 on Page 374.

The solutions to Problems 21.8 on Page 384 and 21.15 on Page 390 also illustrate the use of parallelism.

Caching

Caching is a great tool whenever computations are repeated. For example, the central idea behind dynamic programming is caching results from intermediate computations. Caching is also extremely useful when implementing a service that is expected to respond to many requests over time, and many requests are repeated. Workloads on web services exhibit this property. Solution 20.1 on Page 363 sketches the design of an online spell correction service; one of the key issues is performing cache updates in the presence of concurrent requests. Solution 20.9 on Page 372 shows how multithreading combines with caching in code which tests the Collatz hypothesis.

Scanned by CamScanner

21.1 Design a spell checker

Designing a good spelling correction system can be challenging. We discussed spelling correction in the context of edit distance (Problem 17.2 on Page 299). However, in that problem, we only computed the Levenshtein distance between a pair of strings. A spell checker must find a set of words that are closest to a given word from the entire dictionary. Furthermore, the Levenshtein distance may not be the right distance function when performing spelling correction—it does not take into account the commonly misspelled words or the proximity of letters on a keyboard.

How would you build a spelling correction system?

Hint: Start with an appropriate notion of distance between words.

Solution: The basic idea behind most spelling correction systems is that the misspelled word's Levenshtein distance from the intended word tends to be very small (one or two edits). Hence, if we keep a hash table for all the words in the dictionary and look for all the words that have a Levenshtein distance of 2 from the text, it is likely that the intended word will be found in this set. If the alphabet has m characters and the search text has n characters, we need to perform $O(n^2m^2)$ hash table lookups. More precisely, for a word of length n, we can pick any two characters and change them to any other character in the alphabet. The total number of ways of selecting any two characters is $n(n-1)/2$, and each character can be changed to one of $(m-1)$ other chars. Therefore, the number of lookups is $n(n-1)(m-1)^2/2$.

The intersection of the set of all strings at a distance of two or less from a word and the set of dictionary words may be large. It is important to provide a ranked list of suggestions to the users, with the most likely candidates are at the beginning of the list. There are several ways to achieve this:

- Typing errors model—often spelling mistakes are a result of typing errors. Typing errors can be modeled based on keyboard layouts.
- Phonetic modeling—a big class of spelling errors happen when the person spelling it knows how the words sounds but does not know the exact spelling. In such cases, it helps to map the text to phonemes and then find all the words that map to the same phonetic sequence.
- History of refinements—often users themselves provide a great amount of data about the most likely misspellings by first entering a misspelled word and then correcting it. This historic data is often immensely valuable for spelling correction.
- Stemming—often the size of a dictionary can be reduced by keeping only the stemmed version of each word. (This entails stemming the query text.)

21.2 Design a solution to the stemming problem

When a user submits the query "computation" to a search engine, it is quite possible he might be interested in documents containing the words "computers", "compute", and "computing" also. If you have several keywords in a query, it becomes difficult to search for all combinations of all variants of the words in the query.

Scanned by CamScanner

Stemming is the process of reducing all variants of a given word to one common root, both in the query string and in the documents. An example of stemming would be mapping {*computers*, *computer*, *compute*} to *compute*. It is almost impossible to succinctly capture all possible variants of all words in the English language but a few simple rules can get us most cases.

Design a stemming algorithm that is fast and effective.

Hint: The examples are suggestive of general rules.

Solution: Stemming is a large topic. Here we mention some basic ideas related to stemming—this is in no way a comprehensive discussion on stemming approaches.

Most stemming systems are based on simple rewrite rules, e.g., remove suffixes of the form "es", "s", and "ation". Suffix removal does not always work. For example, wolves should be stemmed to wolf. To cover this case, we may have a rule that replaces the suffix "ves" with "f".

Most rules amount to matching a set of suffixes and applying the corresponding transformation to the string. One way of efficiently performing this is to build a finite state machine based on all the rules.

A more sophisticated system might have exceptions to the broad rules based on the stem matching some patterns. The Porter stemmer, developed by Martin Porter, is considered to be one of the most authoritative stemming algorithms in the English language. It defines several rules based on patterns of vowels and consonants.

Other approaches include the use of stochastic methods to learn rewrite rules and n-gram based approaches where we look at the surrounding words to determine the correct stemming for a word.

21.3 Plagiarism detector

Design an efficient algorithm that takes as input a set of text files and returns pairs of files which have substantial commonality.

Hint: Design a hash function which can incrementally hash $S[i : i + k - 1]$ for $i = 0, 1, 2, \ldots$.

Solution: We will treat each file as a string. We take a pair of files as having substantial commonality if they have a substring of length k in common, where k is a measure of commonality. (Later, we will have a deeper discussion as to the validity of this model.)

Let l_i be the length of the string corresponding to the ith file. For each such string we can compute $l_i - k + 1$ hash codes, one for *each* k length substring.

We insert these hash codes in a hash table G, recording which file each code corresponds to, and the offset the corresponding substring appears at. A collision indicates that the two length-k substrings are potentially the same.

Since we are computing hash code for each k length substring, it is important for efficiency to have a hash function which can be updated incrementally when we delete one character and add another. Solution 7.14 on Page 107 describes such a hash function.

Scanned by CamScanner

In addition, it is important to have many slots in G, so that collisions of unequal strings is rare. A total of $\sum_{i=1}^{|S|}(l_i - k + 1)$ strings are added to the hash table. If k is small relative to the string length and G has significantly fewer slots than the total number of characters in the strings, then we are certain to have collisions.

If it is not essential to return an exact answer, we can save storage by only considering a subset of substrings, e.g., those whose hash codes have 0s in the last b bits. This means that on average we consider $\frac{1}{2^b}$ of the total set of substrings (assuming the hash function does a reasonable job of spreading keys).

The solution presented above can lead to many false positives. For example, if each file corresponds to an HTML page, all pages with a common embedded script of length k or longer will show up as having substantial overlap. We can account for this through preprocessing, e.g., parsing the documents and removing headers. (This may require multiple passes with some manual inspection driving the process.) This solution will also not work if, for example, the files are similar in that they are the exact same program, but with ids renamed. (It is however, resilient to code blocks being moved around.) By normalizing ids, we can cast the problem back into one of finding common substrings.

The approach we have just described is especially effective when the number of files is very large and the files are spread over many servers. In particular, the map-reduce framework can be used to achieve the effect of a single G spread over many servers.

21.4 Pair users by attributes

You are building a social network where each user specifies a set of attributes. You would like to pair each user with another unpaired user that specified the same set of attributes.

You are given a sequence of users where each user has a unique 32-bit integer key and a set of attributes specified as strings. When you read a user, you should pair that user with another previously read user with identical attributes who is currently unpaired, if such a user exists. If the user cannot be paired, you should add him to the unpaired set.

Hint: Map sets of attributes to strings.

Solution: A brute-force approach is to compare each new user's attributes with the attributes of users in the unpaired set. This leads to $O(n^2)$ time complexity, where n is the number of users.

To improve the time complexity, we need a way to find users who have the same attributes as the new user. A hash table whose keys are subsets of attributes and values are users is the perfect choice. This leaves us with the problem of designing a hash function which is suitable for subsets of attributes.

If the total number of possible attributes is small, we can represent a subset of attributes with a bit array, where each index represents a specific attribute. Specifically, we store a 1 at an index if that attribute is present. We can then use any hash function

Scanned by CamScanner

for bit arrays. The time complexity to process the n users is a hash lookup per user, i.e., $O(nm)$, where m is the number of distinct attributes. The space complexity is also $O(nm)$—n entries, each of size m.

If the set of possible attributes is large, and most users have a small subset of those attributes, the bit vector representation for subsets is inefficient from both a time and space perspective. A better approach to represent sparse subsets is directly record the elements. To ensure equal subsets have equal hash codes, we need to represent the subsets in a unique way. One approach is to sort the elements. We can view the sorted sequence of attributes as a single string, namely the concatenation the individual elements. We then use a hash function for strings. For example, if the possible attributes are {USA, Senior, Income, Prime Customer}, and a user has attributes {USA, Income}, we represent his set of attributes as the string "Income,USA".

The time complexity of this approach is $O(M)$, where M is the sum of the sizes of the attribute sets for all users.

21.5 Design a system for detecting copyright infringement

YouTV.com is a successful online video sharing site. Hollywood studios complain that much of the material uploaded to the site violates copyright.

Design a feature that allows a studio to enter a set V of videos that belong to it, and to determine which videos in the YouTV.com database match videos in V.

Hint: Normalize the video format and create signatures.

Solution: If we replaced videos everywhere with documents, we could use the techniques in Solution 21.3 on Page 379, where we looked for near duplicate documents by computing hash codes for each length-k substring.

Videos differ from documents in that the same content may be encoded in many different formats, with different resolutions, and levels of compression.

One way to reduce the duplicate video problem to the duplicate document problem is to re-encode all videos to a common format, resolution, and compression level. This in itself does not mean that two videos of the same content get reduced to identical files—the initial settings affect the resulting videos. However, we can now "signature" the normalized video.

A trivial signature would be to assign a 0 or a 1 to each frame based on whether it has more or less brightness than average. A more sophisticated signature would be a 3 bit measure of the red, green, and blue intensities for each frame. Even more sophisticated signatures can be developed, e.g., by taking into account the regions on individual frames. The motivation for better signatures is to reduce the number of false matches returned by the system, and thereby reduce the amount of time needed to review the matches.

The solution proposed above is algorithmic. However, there are alternative approaches that could be effective: letting users flag videos that infringe copyright (and possibly rewarding them for their effort), checking for videos that are identical to

Scanned by CamScanner

videos that have previously been identified as infringing, looking at meta-information in the video header, etc.

Variant: Design an online music identification service.

21.6 Design TEX

The TEX system for typesetting beautiful documents was designed by Don Knuth. Unlike GUI based document editing programs, TEX relies on a markup language, which is compiled into a device independent intermediate representation. TEX formats text, lists, tables, and embedded figures; supports a very rich set of fonts and mathematical symbols; automates section numbering, cross-referencing, index generation; exports an API; and much more.

How would you implement TEX?

Hint: There are two aspects—building blocks (e.g., fonts, symbols) and hierarchical layout.

Solution: Note that the problem does not ask for the design of TEX, which itself is a complex problem involving feature selection, and language design. There are a number of issues common to implementing any such program: programming language selection, lexing and parsing input, error handling, macros, and scripting.

Two key implementation issues specific to TEX are a specifying fonts and symbols (e.g., A, b, f, $\sum$, $\oint$, $\uplus$), and assembling a document out of components.

Focusing on the second aspect, a reasonable abstraction is to use a rectangular bounding box to describe components. The description is hierarchical: each individual symbol is a rectangle, lines and paragraphs are made out of these rectangles and are themselves rectangles, as are section titles, tables and table entries, and included images. A key algorithmic problem is to assemble these rectangles, while preserving hard constraints on layout, and soft constraints on aesthetics. See also Problem 17.11 on Page 318 for an example of the latter.

Turning our attention to symbol specification, the obvious approach is to use a 2D array of bits to represent each symbol. This is referred to as a bit-mapped representation. The problem with bit-mapped fonts is that the resolution required to achieve acceptable quality is very high, which leads to huge documents and font-libraries. Different sizes of the same symbol need to be individually mapped, as do italicized and bold-face versions.

A better approach is to define symbols using mathematical functions. A reasonable approach is to use a language that supports quadratic and cubic functions, and elementary graphics transformations (rotation, interpolation, and scaling). This approach overcomes the limitations of bit-mapped fonts—parameters such as aspect ratio, font slant, stroke width, serif size, etc. can be programmed.

Other implementation issues include enabling cross-referencing, automatically creating indices, supporting colors, and outputting standard page description formats (e.g., PDF)).

Donald Knuth's book "*Digital Typography*" describes in great detail the design and implementation of TEX.

Scanned by CamScanner

Keyword-based search engines maintain a collection of several billion documents. One of the key computations performed by a search engine is to retrieve all the documents that contain the keywords contained in a query. This is a nontrivial task in part because it must be performed in a few tens of milliseconds.

Here we consider a smaller version of the problem where the collection of documents can fit within the RAM of a single computer.

Given a million documents with an average size of 10 kilobytes, design a program that can efficiently return the subset of documents containing a given set of words.

Hint: Build on the idea of a book's index.

Solution: The predominant way of doing this is to build inverted indices. In an inverted index, for each word, we store a sequence of locations where the word occurs. The sequence itself could be represented as an array or a linked list. Location is defined to be the document ID and the offset in the document. The sequence is stored in sorted order of locations (first ordered by document ID, then by offset). When we are looking for documents that contain a set of words, what we need to do is find the intersection of sequences for each word. Since the sequences are already sorted, the intersection can be done in time proportional to the aggregate length of the sequences. We list a few optimizations below.

- *Compression*—compressing the inverted index helps both with the ability to index more documents as well as memory locality (fewer cache misses). Since we are storing sorted sequences, one way of compressing is to use delta compression where we only store the difference between the successive entries. The deltas can be represented in fewer bits.
- *Caching*—the distribution of queries is usually fairly skewed and it helps a great deal to cache the results of some of the most frequent queries.
- *Frequency-based optimization*—since search results often do not need to return every document that matches (only top ten or so), only a fraction of highest quality documents can be used to answer most of the queries. This means that we can make two inverted indices, one with the high quality documents that stays in RAM and one with the remaining documents that stays on disk. This way if we can keep the number of queries that require the secondary index to a small enough number, then we can still maintain a reasonable throughput and latency.
- *Intersection order*—since the total intersection time depends on the total size of sequences, it would make sense to intersect the words with smaller sets first. For example, if we are looking for "USA GDP 2009", it would make sense to intersect the lists for GDP and 2009 before trying to intersect the sequence for USA.

We could also build a multilevel index to improve accuracy on documents. For a high priority web page, we can decompose the page into paragraphs and sentences, which are indexed individually. That way the intersections for the words might be within the same context. We can pick results with closer index values from these sequences.

Scanned by CamScanner

See the sorted array intersection problem 14.1 on Page 228 and digest problem 13.8 on Page 210 for related issues.

21.8 Implement PageRank

The PageRank algorithm assigns a rank to a web page based on the number of "important" pages that link to it. The algorithm essentially amounts to the following:

(1.) Build a matrix A based on the hyperlink structure of the web. Specifically, $A_{ij} = \frac{1}{d_j}$ if page j links to page i; here d_j is the number of distinct pages linked from page j.

(2.) Find X satisfying $X = \epsilon[1] + (1 - \epsilon)AX$. Here ϵ is a constant, e.g., $\frac{1}{7}$, and $[1]$ represents a column vector of 1s. The value $X[i]$ is the rank of the ith page.

The most commonly used approach to solving the above equation is to start with a value of X, where each component is $\frac{1}{n}$ (where n is the number of pages) and then perform the following iteration: $X_k = \epsilon[1] + (1 - \epsilon)AX_{k-1}$.

Design a system that can compute the ranks of ten billion web pages in a reasonable amount of time.

Hint: This must be performed on an ensemble of machines. The right data structures will simplify the computation.

Solution: Since the web graph can have billions of vertices and it is mostly a sparse graph, it is best to represent the graph as an adjacency list. Building the adjacency list representation of the graph may require a significant amount of computation, depending upon how the information is collected. Usually, the graph is constructed by downloading the pages on the web and extracting the hyperlink information from the pages. Since the URL of a page can vary in length, it is often a good idea to represent the URL by a hash code.

The most expensive part of the PageRank algorithm is the repeated matrix multiplication. Usually, it is not possible to keep the entire graph information in a single machine's RAM. Two approaches to solving this problem are described below.

- Disk-based sorting—we keep the column vector X in memory and load rows one at a time. Processing Row i simply requires adding $A_{i,j}X_j$ to X_j for each j such that $A_{i,j}$ is not zero. The advantage of this approach is that if the column vector fits in RAM, the entire computation can be performed on a single machine. This approach is slow because it uses a single machine and relies on the disk.
- Partitioned graph—we use n servers and partition the vertices (web pages) into n sets. This partition can be computed by partitioning the set of hash codes in such a way that it is easy to determine which vertex maps to which machine. Given this partitioning, each machine loads its vertices and their outgoing edges into RAM. Each machine also loads the portion of the PageRank vector corresponding to the vertices it is responsible for. Then each machine does a local matrix multiplication. Some of the edges on each machine may correspond to vertices that are owned by other machines. Hence the result vector contains nonzero entries for vertices that are not owned by the local machine. At the end

Scanned by CamScanner

of the local multiplication it needs to send updates to other hosts so that these values can be correctly added up. The advantage of this approach is that it can process arbitrarily large graphs.

PageRank runs in minutes on a single machine on the graph consisting of the six million pages that constitute Wikipedia. It takes roughly 70 iterations to converge on this graph. Anecdotally, PageRank takes roughly 200 iterations to converge on the web graph.

21.9 Design a scalable priority system

Maintaining a set of prioritized jobs in a distributed system can be tricky. Applications include a search engine crawling web pages in some prioritized order, as well as event-driven simulation in molecular dynamics. In both cases the number of jobs is in the billions and each has its own priority.

Design a system for maintaining a set of prioritized jobs that implements the following API: (1.) insert a new job with a given priority; (2.) delete a job; (3.) find the highest priority job. Each job has a unique ID. Assume the set cannot fit into a single machine's memory.

Hint: How would you partition jobs across machines? Is it always essential to operate on the highest priority job?



Solution: If we have enough RAM on a single machine, the most simple solution would be to maintain a min-heap where entries are ordered by their priority. An additional hash table can be used to map jobs to their corresponding entry in the min-heap to make deletions fast.

A more scalable solution entails partitioning the problem across multiple machines. One approach is to apply a hash function to the job ids and partition the resulting hash codes into ranges, one per machine. Insert as well as delete require communication with just one server. To do extract-min, we send a lookup minimum message to all the machines, infer the min from their responses, and then delete it.

At a given time many clients may be interested in the highest priority event, and it is challenging to distribute this problem well. If many clients are trying to do this operation at the same time, we may run into a situation where most clients will find that the min event they are trying to extract has already been deleted. If the throughput of this service can be handled by a single machine, we can make one server solely responsible for responding to all the requests. This server can prefetch the top hundred or so events from each of the machines and keep them in a heap.

In many applications, we do not need strong consistency guarantees. We want to spend most of our resources taking care of the highest priority jobs. In this setting, a client could pick one of the machines at random, and request the highest priority job. This would work well for the distributed crawler application. It is not suited to event-driven simulation because of dependencies.

Scanned by CamScanner

21.10 Create photomosaics

A photomosaic is built from a collection of images called "tiles" and a target image. The photomosaic is another image which approximates the target image and is built by juxtaposing the tiles. Quality is defined by human perception.

Design a program that produces high quality mosaics with minimal compute time.

Hint: How would you define the distance between two images?

Solution: A good way to begin is to partition the image into $s \times s$-sized squares, compute the average color of each such image square, and then find the tile that is closest to it in the color space. Distance in the color space can be the $L2$-distance over the Red-Green-Blue (RGB) intensities for the color. As you look more carefully at the problem, you might conclude that it would be better to match each tile with an image square that has a similar structure. One way could be to perform a coarse pixelization (2×2 or 3×3) of each image square and finding the tile that is "closest" to the image square under a distance function defined over all pixel colors. In essence, the problem reduces to finding the closest point from a set of points in a k-dimensional space.

Given m tiles and an image partitioned into n squares, then a brute-force approach would have $O(mn)$ time complexity. You could improve on this by first indexing the tiles using an appropriate search tree. You can also run the matching in parallel by partitioning the original image into subimages and searching for matches on the subimages independently.

21.11 Implement Mileage Run

Airlines often give customers who fly frequently with them a "status". This status allows them early boarding, more baggage, upgrades to executive class, etc. Typically, status is a function of miles flown in the past twelve months. People who travel frequently by air sometimes want to take a round trip flight simply to maintain their status. The destination is immaterial—the goal is to minimize the cost-per-mile (cpm), i.e., the ratio of dollars spent to miles flown.

Design a system that will help its users find mileage runs.

Hint: Partition the implied features into independent tasks.

Solution: There are two distinct aspects to the design. The first is the user-facing portion of the system. The second is the server backend that gets flight-price-distance information and combines it with user input to generate the alerts.

We begin with the user-facing portion. For simplicity, we illustrate it with a web-app, with the realization that the web-app could also be written as a desktop or mobile app. The web-app has the following components: a login page, a manage alerts page, a create an alert page, and a results page. For such a system we would like defer to a single-sign-on login service such as that provided by Google or Facebook. The management page would present login information, a list of alerts, and the ability to create an alert.

Scanned by CamScanner

One reasonable formulation of an alert is that it is an origin city, a target cpm, and optionally, a date or range of travel dates. The results page would show flights satisfying the constraints. Note that other formulations are also possible, such as how frequently to check for flights, a set of destinations, a set of origins, etc.

The classical approach to implement the web-app front end is through dynamically generated HTML on the server, e.g., through Java Server Pages. It can be made more visually appealing and intuitive by making appropriate use of cascaded style sheets, which are used for fonts, colors, and placements. The UI can be made more efficient through the use of Javascript to autocomplete common fields, and make attractive date pickers.

Modern practice is to eschew server-side HTML generation, and instead have a single-page application, in which Javascript reads and writes JavaScript Object Notation (JSON) objects to the server, and incrementally updates the single-page based. The AngularJS framework supports this approach.

The web-app backend server has four components: gathering flight data, matching user-generated alerts to this data, persisting data and alerts, and generating the responses to browser initiated requests.

Flight data can be gathered via "scraping" or by subscribing to a flight data service. Scraping refers to extraction of data from a website. It can be quite involved—some of the issues are parsing the results from the website, filling in form data, and running the Javascript that often populates the actual results on a page. Selenium is a Java library that can programmatically interface to the Firefox browser, and is appropriate for scraping sites that are rich in Javascript. Most flight data services are paid. ITA software provides a very widely used paid aggregated flight data feed service. The popular Kayak site provides an Extensible Markup Language (XML) feed of recently discovered fares, which can be a good free alternative. Flight data does not include the distance between airports, but there are websites which return the distance between airport codes which can be used to generate the cpm for a flight.

There are a number of common web application frameworks—essentially libraries that handle many common tasks—that can be used to generate the server. Java and Python are very commonly used for writing the backend for web applications.

Persistence of data can be implemented through a database. Most web application frameworks provide support for automating the process of reading and writing objects from and to a database. Finally, web application frameworks can route incoming HTTP requests to appropriate code—this is through a configuration file matching URLs to methods. The framework provides convenience methods for accessing HTTP fields and writing results. Frameworks also provide HTTP templating mechanisms, wherein developers intersperse HTML with snippets of code that dynamically add content to the HTML.

Web application frameworks typically implement cron functionality, wherein specified functions are executed at a regular interval. This can be used to periodically scrape data and check if the condition of an alert is matched by the data.

Finally, the web app can be deployed via a platform-as-a-service such as Amazon Web Services and Google App Engine.

Scanned by CamScanner

21.12 Implement Connexus

How would you design Connexus, a system by which users can share pictures? Address issues around access control (public, private), picture upload, organizing pictures, summarizing sets of pictures, allowing feedback, and displaying geographical and temporal information for pictures.

Hint: Think about the UI, in particular UI widgets that make for an engaging product.

Solution: There are three aspects to Connexus. The first is the server backend, used to store images, and meta-data, such as author, comments, hashtags, GPS coordinates, etc., as well as run `cron` jobs to identify trending streams. The technology for this is similar to Mileage Run (Solution 21.11 on Page 386), with the caveat that a database is not suitable for storing large binary objects such as images.

The web UI is also similar to Mileage Run, with a login page, a management page, and pages for displaying images. Images can be grouped based on a concept of a stream, with comment boxes annotating streams. Facebook integration would make it easier to share links to streams, and post new images as status updates. Search capability and discussion boards also enhance the user experience.

Some UI features that are especially appealing are displaying images by location on a zoomable map, slider UI controls to show subsets of images in a stream based on the selected time intervals, a file upload dialog with progress measures, support for multiple simultaneous uploads, and drag-and-drop upload. All these UI widgets are provided by, for example, the jQuery-UI Javascript library. (This library also makes the process of creating autocompletion on text entry fields trivial.)

Connexus is an application that begs for a mobile client. A smartphone provides a camera, location information, and push notifications. These can be used to make it easier to create and immediately upload geo-tagged images, find nearby images, and be immediately notified on updates and comments. The two most popular mobile platforms, iOS and Android, have APIs rich in UI widgets and media access. Both can use serialization formats such as JSON or protocol buffers to communicate with the server via remote procedure calls layered over HTTP.

21.13 Design an online advertising system

Jingle, a search engine startup, has been very successful at providing a high-quality Internet search service. A large number of customers have approached Jingle and asked it to display paid advertisements for their products and services alongside search results.

Design an advertising system for Jingle.

Hint: Consider the stakeholders separately.

Solution: Reasonable goals for such a system include

- providing users with the most relevant ads,
- providing advertisers the best possible return on their investment, and
- minimizing the cost and maximizing the revenue to Jingle.

Scanned by CamScanner

Two key components for such a system are:

- The front-facing component, which advertisers use to create advertisements, organize campaigns, limit when and where ads are shown, set budgets, and create performance reports.
- The ad-serving system, which selects which ads to show on the searches.

The front-facing system can be a fairly conventional web application, i.e., a set of web pages, middleware that responds to user requests, and a database. Key features include:

- User authentication—a way for users to create accounts and authenticate themselves. Alternatively, use an existing single sign-on login service, e.g., Facebook or Google.
- User input—a set of form elements to let advertisers specify ads, advertising budget, and search keywords to bid on.
- Performance reports—a way to generate reports on how the advertiser's money is being spent.
- Customer service—even the best of automated systems require occasional human interaction, e.g., ways to override limits on keywords. This requires an interface for advertisers to contact customer service representatives, and an interface for those representatives to interact with the system.

The whole front-end system can be built using, for example, HyperText Markup Language (HTML) and JavaScript. A commonly used approach is to use a LAMP stack on the server-side: Linux as the OS, Apache as the HTTP server, MySQL as the database software, and PHP for the application logic.

The ad-serving system is less conventional. The ad-serving system would build a specialized data structure, such as a decision tree, from the ads database. It chooses ads from the database of ads based on their "relevance" to the search. In addition to keywords, the ad-serving systems can use knowledge of the user's search history, how much the advertiser is willing to pay, the time of day, user locale, and type of browser. Many strategies can be envisioned here for estimating relevance, such as, using information retrieval or machine learning techniques that learn from past user interactions.

The ads could be added to the search results by embedding JavaScript in the results page. This JavaScript pulls in the ads from the ad-serving system directly. This helps isolate the latency of serving search results from the latency of serving ad results.

21.14 Design a recommendation system

Jingle wants to generate more page views on its news site. A product manager has the idea to add to each article a sidebar of clickable snippets from articles that are likely to be of interest to someone reading the current article.

Design a system that automatically generates a sidebar of related articles.

Hint: This problem can be solved with various degrees of algorithmic sophistication: none at all, simple frequency analysis, or machine learning.

Scanned by CamScanner

Solution: The key technical challenge in this problem is to come up with the list of articles—the code for adding these to a sidebar is trivial.

One suggestion might be to add articles that have proved to be popular recently. Another is to have links to recent news articles. A human reader at Jingle could tag articles which he believes to be significant. He could also add tags such as finance, sports, and politics, to the articles. These tags could also come from the HTML meta-tags or the page title.

We could also provide randomly selected articles to a random subset of readers and see how popular these articles prove to be. The popular articles could then be shown more frequently.

On a more sophisticated level, Jingle could use automatic textual analysis, where a similarity is defined between pairs of articles—this similarity is a real number and measures how many words are common to the two. Several issues come up, such as the fact that frequently occurring words such as "for" and "the" should be ignored and that having rare words such as "arbitrage" and "diesel" in common is more significant than having say, "sale" and "international".

Textual analysis has problems, such as the fact that two words may have the same spelling but completely different meanings (anti-virus means different things in the context of articles on acquired immune deficiency syndrome (AIDS) and computer security). One way to augment textual analysis is to use collaborative filtering—using information gleaned from many users. For example, by examining cookies and timestamps in the web server's log files, we can tell what articles individual users have read. If we see many users have read both A and B in a single session, we might want to recommend B to anyone reading A. For collaborative filtering to work, we need to have many users.

21.15 Design an optimized way of distributing large files

Jingle is developing a search feature for breaking news. New articles are collected from a variety of online news sources such as newspapers, bulletin boards, and blogs, by a single lab machine at Jingle. Every minute, roughly one thousand articles are posted and each article is 100 kilobytes.

Jingle would like to serve these articles from a data center consisting of a 1000 servers. For performance reasons, each server should have its own copy of articles that were recently added. The data center is far away from the lab machine.

Design an efficient way of copying one thousand files each 100 kilobytes in size from a single lab server to each of 1000 servers in a distant data center.

Hint: Exploit the data center.

Solution: Assume that the bandwidth from the lab machine is a limiting factor. It is reasonable to first do trivial optimizations, such as combining the articles into a single file and compressing this file.

Scanned by CamScanner

Opening 1000 connections from the lab server to the 1000 machines in the data center and transferring the latest news articles is not feasible since the total data transferred will be approximately 100 gigabytes (without compression).

Since the bandwidth between machines in a data center is very high, we can copy the file from the lab machine to a single machine in the data center and have the machines in the data center complete the copy. Instead of having just one machine serve the file to the remaining 999 machines, we can have each machine that has received the file initiate copies to the machines that have not yet received the file. In theory, this leads to an exponential reduction in the time taken to do the copy.

Several additional issues have to be dealt with. Should a machine initiate further copies before it has received the entire file? (This is tricky because of link or server failures.) How should the knowledge of machines which do not yet have copies of the file be shared? (There can be a central repository or servers can simply check others by random selection.) If the bandwidth between machines in a data center is not a constant, how should the selections be made? (Servers close to each other, e.g., in the same rack, should prefer communicating with each other.)

Finally, it should be mentioned that there are open source solutions to this problem, e.g., Unison and BitTorrent, which would be a good place to start.



21.16 Design the World Wide Web

Design the World Wide Web. Specifically, describe what happens when you enter a URL in a browser address bar, and press return.

Hint: Follow the flow of information.

Solution: At the network level, the browser extracts the domain name component of the URL, and determines the IP address of the server, e.g., through a call to a Domain Name Server (DNS), or a cache lookup. It then communicates using the HTTP protocol with the server. HTTP itself is built on top of TCP/IP, which is responsible for routing, reassembling, and resending packets, as well as controlling the transmission rate.

The server determines what the client is asking for by looking at the portion of the URL that comes after the domain name, and possibly also the body of the HTTP request. The request may be for something as simple a file, which is returned by the webserver; HTTP spells out a format by which the type of the returned file is specified. For example, the URL http://go.com/imgs/abc.png may encode a request for the file whose hierarchical name is imgs/abc.png relative to a base directory specified at configuration to the web server.

The URL may also encode a request to a service provided by the web server. For example, http://go.com/lookup/flight?num=UA37,city=AUS is a request to the lookup/flight service, with an argument consisting of two attribute-value pair. The service could be implemented in many ways, e.g., Java code within the server, or a Common Gateway Interface (CGI) script written in Perl. The service generates a HTTP response, typically HTML, which is then returned to the browser. This response could encode data which

Scanned by CamScanner

is used by scripts running in the browser. Common data formats include JSON and XML.

The browser is responsible for taking the returned HTML and displaying it on the client. The rendering is done in two parts. First, a parse tree (the DOM) is generated from the HTML, and then a rendering library "paints" the screen. The returned HTML may include scripts written in JavaScript. These are executed by the browser, and they can perform actions like making requests and updating the DOM based on the responses—this is how a live stock ticker is implemented. Styling attributes (CSS) are commonly used to customize the look of a page.

Many more issues exist on both the client and server side: security, cookies, HTML form elements, HTML styling, and handlers for multi-media content, to name a few.

21.17 Estimate the hardware cost of a photo sharing app

Estimate the hardware cost of the server hardware needed to build a photo sharing app used by every person on the earth.

Hint: Use variables to denote quantities and costs, and relate them with equations. Then fill in reasonable values.

Solution: The cost is a function of server CPU and RAM resources, storage, and bandwidth, as well as the number and size of the images that are uploaded each day. We will create an estimate based on unit costs for each of these. We assume a distributed architecture in which images are spread ("sharded") across servers.

Assume each user uploads i images each day with an average size of s bytes, and that each image is viewed v times. After d days, the storage requirement is $isdN$, where N is the number of users. Assuming $v \gg 1$, i.e., most images are seen many times, the server cost is dominated by the time to serve the images. The servers are required to serve up Niv images and $Nivs$ bytes each day. Assuming a server can handle h HTTP requests per second and has an outgoing bandwidth of b bytes per second, the number of required servers is $\max(Niv/Th, Nivs/Tb)$, where T is the number of seconds in a day.

Reasonable values for N, i, s, and v are 10^{10}, 10, 10^5, and 100. Reasonable values for h and b are 10^4 and 10^8. There are approximately 10^5 seconds in a day. Therefore the number of servers required is $\max((10^{10} \times 10 \times 100)/(10^5 \times 10^4), (10^{10} \times 10 \times 100 \times 10^5)/(10^5 \times 10^8)) = 10^5$. Each server would cost \$1000, so the total cost would be of the order of 100 million dollars.

Storage costs are approximately \$0.1 per gigabyte, and we add $Nis = 10^{10} \times 10 \times 10^5$ bytes each day, so each day we need to add a million dollars worth of storage.

The above calculation leaves out many costs, such as electricity, cooling, and network. It also neglects computations such as computing trending data and spam analysis. Furthermore, there is no measure of the cost of redundancy, such as replicated storage, or the ability to handle nonuniform loads. Nevertheless, it is a decent starting point. What is remarkable is the fact that the entire world can be connected through images at a very low cost—pennies per person.

Scanned by CamScanner